



ROBOCUPJUNIOR RESCUE MAZE 2026

TEAM DESCRIPTION PAPER

Orange Bots

Abstract

The Orange Bots team has developed an autonomous robot specifically tailored for the RoboCupJunior Rescue Maze competition. Our system employs a robust hybrid computing architecture, leveraging an ESP32 microcontroller for low-level, real-time sensor control, paired with a Raspberry Pi 4 for high-level logic, navigation, and decision-making. The robot is equipped with a comprehensive sensor suite designed for complex maze navigation, including ultrasonic sensors for obstacle detection, an Inertial Measurement Unit (IMU) for accurate orientation tracking, motor encoders for precise movement monitoring, and a Raspberry Pi Camera Module 3 for victim identification. Navigation is driven by a Depth-First Search (DFS) algorithm to ensure the exhaustive and systematic exploration of the maze environment. Inter-subsystem communication is achieved via a reliable serial interface. The primary objective of our team is to construct a dependable rescue robot while gaining invaluable hands-on experience in competitive robotics and complex system integration.

1. Introduction

a. Team

David Mónica: 12th grade, Professional Electrotechnics Course. David is the lead hardware engineer, responsible for the mechanical assembly, electronic wiring, and configuring the communication interfaces between the diverse hardware components.

João Castro: 11th grade, Professional Informatics Programming Course. João is the lead software developer, responsible for programming the high-level software architecture, implementing the DFS navigation logic, and developing the computer vision algorithms for victim detection.

2. Project Planning

a. Overall Project Plan

The core objective of the project was to participate in the RoboCupJunior competition to gain practical experience in the field of robotics. The development followed a linear and straightforward methodology:

1. Hardware Assembly: Constructing the physical chassis and drivetrain.
2. Sensor Integration: Mounting and wiring the electronic components.
3. Control Implementation: Developing the low-level motor control and sensor reading scripts.
4. System Integration: Merging the low-level and high-level systems to enable autonomous operation.

Throughout the development process, the team encountered constraints regarding development time and the availability of specific hardware components, which fundamentally guided the simplified, pragmatic design choices made for the final robot.

b. Integration Plan

The robot's system is structurally organized into two highly specialized and well-defined computational layers:

- Low-Level Layer (ESP32): Tasked with strict real-time operations, this layer handles direct motor control (PWM), processes interrupt-driven encoder data, reads distance metrics from ultrasonic sensors, and acquires orientation data from the IMU.
- High-Level Layer (Raspberry Pi 4): Assumes responsibility for computationally heavy tasks. It executes the core Depth-First Search (DFS) pathfinding algorithm, makes autonomous navigational decisions, and runs image processing pipelines for the onboard camera.

Communication between the two microcomputers is facilitated through a direct serial interface, allowing for a continuous, bidirectional exchange of data: the ESP32 feeds sensor telemetry to the Pi, while the Pi sends directional commands back to the ESP32.

3. Hardware

a. Mechanical Design and Manufacturing

The robot utilizes a four-wheel-drive (4WD) base, with each motor equipped with dedicated rotary encoders. The mechanical structure was deliberately engineered to maintain a low center of gravity and ensure constant traction with the competition surface. This design choice maximizes stability and minimizes slippage, which is critical for maintaining accurate odometry and precise movements within the confined spaces of the maze environment.

b. Electronic Design and Manufacturing

The electronic architecture relies on a master-slave configuration between the processors and interfaces with a variety of digital and analog peripherals.

Component Category	Specific Hardware Used	Primary Function
Main Controllers	ESP32 & Raspberry Pi 4	Real-time peripheral interfacing (ESP32) and high-level logic/vision processing (Raspberry Pi).
Proximity & Ranging	Ultrasonic Sensors	Detecting walls and obstacles for collision avoidance and maze mapping.
Orientation	IMU Sensor	Providing angular velocity and linear acceleration data to maintain heading.
Odometry	Motors with Encoders	Tracking wheel rotations for precise distance measurement and turning execution.
Vision & Detection	Raspberry Pi Camera Module 3	Capturing visual data to identify simulated victims within the maze.
Environmental Sensing	Light Sensor (I2C)	Detecting ambient light variations or floor markings if required.

4. Software

a. General software architecture

The software is distinctly segregated to match the hardware topology, creating a modular and easily debuggable codebase:

- **Embedded Firmware (ESP32):** Focuses strictly on hardware abstraction. It continuously polls sensors and adjusts motor speeds based on PID control loops utilizing encoder feedback.

- Application Software (Raspberry Pi): Focuses on state-machine management. It aggregates the raw data, maps the environment, and dictates the robot's next logical action.

b. Innovative solutions

- DFS Algorithmic Navigation: The robot employs a Depth-First Search algorithm to traverse the maze. It treats every physical intersection as a logical node in a graph. The robot systematically explores open paths until it reaches a dead end, at which point it initiates a backtracking sequence to the last known intersection with unexplored paths. This approach mathematically guarantees full coverage of the reachable maze space.
- Victim Detection Pipeline: Utilizing the high-resolution capabilities of the Raspberry Pi Camera Module 3, the software implements custom image processing routines. It analyzes the video feed using color segmentation and contour recognition to identify the specific visual markers that represent victims, isolating them from the background environment.

5. Performance evaluation

Due to stringent time constraints and limited initial resources during the national development phase, exhaustive and formal testing protocols were not fully realized. However, the modular integration of the subsystems was verified. The primary goal for the upcoming European phase is to utilize this established platform to conduct rigorous, metric-driven tests (e.g., mapping accuracy percentages, victim detection true-positive rates) to fine-tune the system's reliability.

6. Conclusion

The Orange Bot represents a streamlined, highly functional solution to the RoboCupJunior Rescue Maze challenge. The system successfully demonstrates the seamless integration of distributed computing hardware and layered software architecture. Despite initial limitations regarding testing timelines, the project establishes a robust, extensible foundation. This foundational work will allow the team to focus on algorithmic optimization and rigorous physical testing as we prepare for the European stage of the competition.